

# Sri Lanka Institute of Information Technology

## Faculty of Computing

SE2012 - Object Oriented Analysis and Design

Prof. Jagath Samarabandu  
Mr. Jeewaka Perera

Year 02 and Semester 01

# Lecture 01

# Introduction to Java

# Objectives

- Brief introduction to Java
- Understanding JDK, JVM, and JRE
- Java documentation (JavaDoc)
- Introduction to Java classes, data types, and control structures
- Comparison of Java control structures to C

# About Coding

To some of us, writing computer programs is a fascinating game. A program is a building of thought. It is costless to build, it is weightless, and it grows easily under our typing hands.

If we are not careful, its size and complexity will grow out of control, confusing even the person who created it. This is the main problem of programming: keeping programs under control.

When a program works, it is beautiful. The art of programming is the skill of controlling complexity. The great program is subdued, made simple in its complexity.

- *Eloquent Javascript* (Marijn Haverbeke)

# Evolution of Java



**1991:** Initiated by James Gosling, Mike Sheridan, and Patrick Naughton at Sun Microsystems



**Project Name:** Originally named "Oak" after an oak tree outside Gosling's office, later renamed "Java"



**1995:** Officially launched as Java 1.0



**1996:** Java Development Kit (JDK) 1.0 released



**1999:** Java 2 (J2SE) introduced



**2004:** Java 5.0 (J2SE 5.0) introduced major updates like generics and metadata



**2006:** Java becomes open source under the GNU General Public License



**2010:** Oracle acquires Sun Microsystems

# Java's Philosophy and Design Goals

## Philosophy:

- "Write Once, Run Anywhere"

## Design Goals:

- **Simplicity:** Easy to learn and use
- **Object-Oriented:** Modular approach to software development
- **Platform Independence:** Runs on any platform with JVM
- **Security:** Built-in security features
- **High Performance:** Efficient execution
- **Robustness:** Strong memory management and error handling

# Java Platform Components



Java Virtual Machine (JVM)



Java Runtime Environment (JRE)



Java Development Kit (JDK)

# Java Virtual Machine - JVM

- **Contains:**

- **Bytecode Interpreter:** Executes Java bytecode.
- **Just-In-Time (JIT) Compiler:** Converts bytecode to native machine code at runtime for improved performance.
- **Garbage Collector:** Manages memory by automatically reclaiming unused objects.

- **Included in:**

- JRE and JDK

# Java Runtime Environment (JRE)



## Contains

**JVM:** To run Java applications.

**Core Libraries:** Essential libraries and classes needed to run Java applications.

**Runtime Tools:** Includes tools for monitoring and debugging applications.



## Download

Visit the [Oracle JRE download page](#).

Choose the appropriate version for your operating system.

Download and install the JRE.

# Java Development Kit (JDK)

## Contains

- Compiler(javac)
  - Converts Java source files (.java) into bytecode files (.class).
  - Essential for developers to compile their code.
- Java Standard Libraries:
  - A vast collection of pre-written classes and methods for various tasks like I/O, networking, utilities, and more.
  - Examples: java.lang, java.util, java.io, java.net.
- Development Tools:
  - **javadoc**: Generates HTML documentation from Java source code.
  - **jdb**: A command-line debugger.
  - **Other tools**: javap (class file disassembler), jar (packaging tool).

## Download

- [Java Downloads | Oracle](#)

# JDK vs. JRE:

**JDK:** For development, includes everything in the JRE plus development tools.

**JRE:** For running Java applications, includes only the JVM and core libraries.

# Java Platforms

## Java SE

- Java SE's API provides the core functionality of the Java programming language.
- Defines everything from the basic types and objects of the Java programming language to high-level classes that are used for networking, security, database access, graphical user interface (GUI) development, and XML parsing.
- In addition to the core API, the Java SE platform consists of a virtual machine, development tools, deployment technologies, and other class libraries and toolkits commonly used in Java technology applications.

## Java EE

- The Java EE platform is built on top of the Java SE platform.
- The Java EE platform provides an API and runtime environment for developing and running large-scale, multi-tiered, scalable, reliable, and secure network applications.

## Java ME

- The Java ME platform provides an API and a small-footprint virtual machine for running Java programming language applications on small devices, like mobile phones.
- The API is a subset of the Java SE API, along with special class libraries useful for small device application development.
- Java ME applications are often clients of Java EE platform services.

## JavaFX

- JavaFX is a platform for creating rich internet applications using a lightweight user-interface API.
- JavaFX applications use hardware-accelerated graphics and media engines to take advantage of higher-performance clients and a modern look-and-feel as well as high-level APIs for connecting to networked data sources.
- JavaFX applications may be clients of Java EE platform services.

# Setting Up



## On Linux

[Installation of the JDK on Linux Platforms \(oracle.com\)](#)



## On Windows

[Installation of the JDK on Microsoft Windows Platforms \(oracle.com\)](#)



## On Mac

[Installation of the JDK on macOS \(oracle.com\)](#)

# Setting Up Java Environment on Windows

## Setting the path temporarily

- set `PATH=%PATH%;C:\Program Files\Java\jdk-11.0.2\bin`

## Permanently Setting the PATH

- Right-click on 'This PC' or 'Computer', then select 'Properties'.
- Click on 'Advanced system settings' and then 'Environment Variables'.
- In the 'System variables' section, find the Path variable and click 'Edit'.
- Add a new entry with the path to the JDK bin directory

# Verifying the Installation



Check the Java  
version:

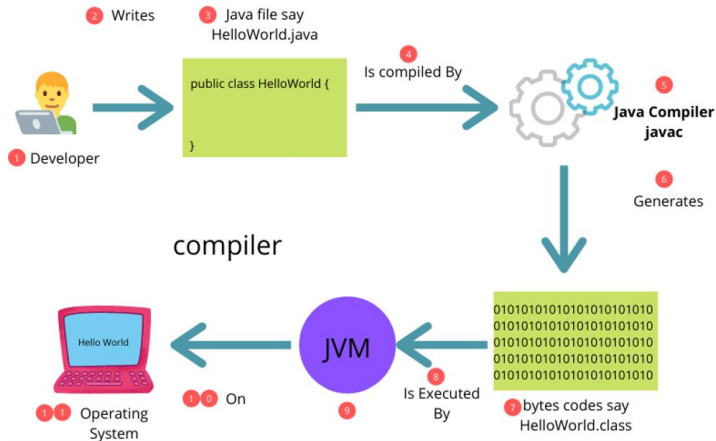
```
java -version
```



Check the Java  
compiler version:

```
javac -version
```

# How to Compile and Execute Java Programs



Tool for generating HTML documentation from Java source code.

Extracts comments marked with special tags and formats them.

## Common JavaDoc Tags

- **@param:** Describes a method parameter.
- **@return:** Describes the return value of a method.
- **@throws:** Documents an exception thrown by a method.
- **@see:** References related information.
- **@since:** Indicates the version when the feature was added.
- **@deprecated:** Marks a method as deprecated.

# Key Words in Java

true false and null are not considered as key words. They are literals and cannot be used as identifiers. [What is an identifier?]

|           |          |            |            |              |
|-----------|----------|------------|------------|--------------|
| abstract  | continue | for        | new        | switch       |
| assert*** | default  | goto*      | package    | synchronized |
| boolean   | do       | if         | private    | this         |
| break     | double   | implements | protected  | throw        |
| byte      | else     | import     | public     | throws       |
| case      | enum**** | instanceof | return     | transient    |
| catch     | extends  | int        | short      | try          |
| char      | final    | interface  | static     | void         |
| class     | finally  | long       | strictfp** | volatile     |
| const*    | float    | native     | super      | while        |

\* not used  
\*\* added in 1.2  
\*\*\* added in 1.4  
\*\*\*\* added in 5.0

# Packages

- **Definition:**

- A package is a namespace that organizes classes and interfaces.
- Helps avoid class name conflicts and control access.

- **Naming Conventions:**

- Package names should be in all lower case.
- java.io
- java.util

[Naming a Package \(The Java™ Tutorials Learning the Java Language Packages\) \(oracle.com\)](#)

# Java Classes



Blueprint for  
creating objects.

More on Objects  
Later.



Defines

attributes (fields/  
variables)  
methods  
(functions).



Classes can contain

Constructors  
Methods  
Fields  
Nested classes.

# Example Class

```
package lecture;
```

```
public class Car {
```

```
    ....
```

```
    ....
```

```
    ....
```

```
}
```

## Defining a Class

- `<AccessModifier> class ClassName{ ... }`

## Naming Conventions

- Class names are generally nouns, in mixed case with the first letter of each internal word capitalized (***CamelCase***).
- Example: `class MyClass { ... }`

# Main Method

```
package lecture;

public class Car {
    ....
    ....
    ....
    public static void main(String[] args) {

    }
}
```

The entry point of any Java application.

It is the method that the JVM calls to start the execution of a program.

**public:** The method is accessible from anywhere.

**static:** The method belongs to the class, not instances of the class.

**void:** The method does not return any value.

**main:** The name of the method that is called by the JVM.

**String[] args:** An array of strings, which stores command-line arguments.

# Naming .java Files

## File Names:

- The file name must exactly match the name of the public class within the file.
- The file name should have a .java extension.

## Example:

- If the class name is **public class Car**, the file name should be **Car.java**.
- If the class name is **public class Student**, the file name should be **Student.java**.

There can only be **one public class per .java file**.

The file name must match the public class name exactly.

Any additional classes in the same file should not be public.

# Compiling and Running a Java File



Compile the program

```
javac FileName.java
```



Run the compiled program

```
java FileName
```

# Activity 01

Write down the steps with necessary commands to compile and execute the below code. Assume that the file has not been saved yet.

```
package lecture01;

public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

# Access Modifiers

**public:** Accessible from any other class.

**protected:** Accessible within the same package and subclasses.

**default (package-private):** Accessible only within the same package.

**private:** Accessible only within the same class.

# Data Types

| Category      | Description                                                 |
|---------------|-------------------------------------------------------------|
| Primitive     | Basic types provided by the language, simple values.        |
| Non-Primitive | Complex types created using classes, arrays, or interfaces. |

# Primitive Data Types

| Data Type | Size   | Range                                                    | Example              |
|-----------|--------|----------------------------------------------------------|----------------------|
| byte      | 8-bit  | -128 to 127                                              | byte b = 10;         |
| short     | 16-bit | -32,768 to 32,767                                        | short s = 1000;      |
| int       | 32-bit | $-2^{31}$ to $2^{31}-1$                                  | int i = 12345;       |
| long      | 64-bit | $-2^{63}$ to $2^{63}-1$                                  | long l = 123456L;    |
| float     | 32-bit | Approximately $\pm 3.40282347E+38F$ (6-7 digits)         | float f = 12.34f;    |
| double    | 64-bit | Approximately $\pm 1.79769313486231570E+308$ (15 digits) | double d = 123.45d;  |
| char      | 16-bit | '\u0000' (0) to '\uffff' (65,535)                        | char c = 'A';        |
| boolean   | 1-bit  | true or false                                            | boolean flag = true; |

# Comparison with C Control Structures

- Java Selection Statements(Similar to c)
  - If, else if, else
  - Switch Statement:
    - Java switch supports String types.
- Java Loops (Similar to c)
  - for
  - while
  - do-while.

# Handling IO in Java

| Feature      | Java                                                                                        | C                                                                                           |
|--------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Basic Print  | <code>System.out.println("Hello, World!");</code>                                           | <code>printf("Hello, World!\n");</code>                                                     |
| Format Print | <code>System.out.printf("Name: %s Age: %d\n", name, age);</code>                            | <code>printf("Name: %s Age: %d\n", name, age);</code>                                       |
| Placeholders | <code>%s</code> for strings, <code>%d</code> for integers, <code>%f</code> for floats, etc. | <code>%s</code> for strings, <code>%d</code> for integers, <code>%f</code> for floats, etc. |

```
public class PrintExample{
    public static void main(String[] args) {
        String name = "John";
        int age = 25;
        double salary = 30000.75;

        // Basic print
        System.out.println("Hello, World!");

        // Formatted print
        System.out.printf("Name: %s Age: %d Salary: %.2f\n",
            name, age, salary);
    }
}
```

# Importing Other Classes from Java Libraries

Use the keyword “import”

Format is

- `import <<package>>.<<ClassName>>;`

This lets you reuse other classes without having to reimplement.

# Handling IO in Java

| Feature      | Java                                                 | C                                                    |
|--------------|------------------------------------------------------|------------------------------------------------------|
| Basic Input  | Scanner scanner = new Scanner(System.in);            | scanf("%d", &number);                                |
| Read Integer | int number = scanner.nextInt();                      | scanf("%d", &number);                                |
| Read String  | String text = scanner.nextLine();                    | scanf("%s", text);                                   |
| Placeholders | %s for strings, %d for integers, %f for floats, etc. | %s for strings, %d for integers, %f for floats, etc. |

```
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Reading an integer
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();

        // Clearing buffer
        scanner.nextLine();

        // Reading a string
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        // Displaying input
        System.out.printf("Name: %s, Number: %d\n", name,
            number);
    }
}
```

## Activity 2

- Write a Java program that determines if a student passes or fails a course based on their score.
- The grading criteria are:
  - If the score is 90 or above, print "Grade A"
  - If the score is between 80 and 89, print "Grade B"
  - If the score is between 70 and 79, print "Grade C"
  - If the score is between 60 and 69, print "Grade D"
  - If the score is below 60, print "Fail"
- Indicate the java filenames and commands for compiling and running the program.

# One-Dimensional Arrays

A one-dimensional array is a list of variables of the same type, accessed by a common name.

## Declaration and Initialization:

- Declaration: `dataType[] arrayName;`
- Initialization: `arrayName = new dataType[size];`
- Combined: `dataType[] arrayName = new dataType[size];`
- Shortcut: `dataType[] arrayName = {value1, value2, ...};`

Arrays are zero-indexed.

Use `arrayName.length` to get the size of the array.

Elements can be accessed using the index notation `arrayName[index]`.

```
public class OneDArrayExample {  
    public static void main(String[] args) {  
        // Declaration and initialization  
        int[] numbers = new int[5];  
        numbers[0] = 10;  
        numbers[1] = 20;  
        numbers[2] = 30;  
        numbers[3] = 40;  
        numbers[4] = 50;  
  
        // Declaration with initialization  
        int[] moreNumbers = {1, 2, 3, 4, 5};  
  
        // Accessing array elements  
        for (int i = 0; i < numbers.length; i++) {  
            System.out.println("Element at index " + i + ": " + numbers[i]);  
        }  
    }  
}
```

# Two-Dimensional Arrays

- Multi-dimensional arrays are arrays of arrays.
- The most common form is the two-dimensional array, which can be thought of as a table with rows and columns.
- Declaration and Initialization:
  - Declaration: `dataType[][] arrayName;`
  - Initialization: `arrayName = new dataType[rows][columns];`
  - Combined: `dataType[][] arrayName = new dataType[rows][columns];`

```
public class MultiDArrayExample {  
    public static void main(String[] args) {  
        // Declaration and initialization  
        int[][] matrix = new int[3][3];  
        matrix[0][0] = 1;  
        matrix[0][1] = 2;  
        matrix[0][2] = 3;  
        matrix[1][0] = 4;  
        matrix[1][1] = 5;  
        matrix[1][2] = 6;  
        matrix[2][0] = 7;  
        matrix[2][1] = 8;  
        matrix[2][2] = 9;  
  
        // Declaration with initialization  
        int[][] anotherMatrix = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
  
        // Accessing array elements  
        for (int i = 0; i < matrix.length; i++) {  
            for (int j = 0; j < matrix[i].length; j++) {  
                System.out.print(matrix[i][j] + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

# Activity 3

- Write a Java program for the following scenario.
- There are 5 students in a class. The students are taking 3 subjects each. Mathematics, Chemistry and Physics. You are tasked with creating a small program to let the teacher enter and view the marks of all these 5 students.
- Indicate the java filenames and commands for compiling and running the program.

# Thank You!